

CloudHunt — Project Plan & Task Breakdown

Project Overview

CloudHunt is a personalized job discovery and matching platform for cloud engineering roles. Users upload a resume, answer a few preference questions, and the app surfaces the best-fit jobs — ranked by relevance — with direct apply links.

Your AWS Stack:

- **S3** — Resume storage + static frontend hosting
 - **Lambda** — Backend logic (parsing, scraping, matching)
 - **DynamoDB** — Jobs, user profiles, application tracking
 - **API Gateway** — REST API layer
 - **EventBridge** — Scheduled job scraping
 - **CloudFront** — CDN for frontend
 - **IAM** — Least-privilege roles and policies
 - **Terraform** — All infrastructure as code
 - **Python (boto3)** — Lambda runtime
 - **React** — Frontend dashboard
-

Phase 1: Project Foundation & Resume Upload

Goal: Set up the repo, Terraform scaffolding, S3 bucket, and a working resume upload flow.

YOU Do (Manually)

#	Task	Why You're Doing This
1	Create the GitHub repo (<code>cloudhunt</code>) with proper <code>.gitignore</code> , branch strategy (<code>main</code> + <code>dev</code>)	You need to own your Git workflow — interviewers check commit history
2	Write <code>terraform/main.tf</code> — provider block, backend config (S3 state bucket + DynamoDB lock table)	Remote state is a real-world pattern, you need to set this up yourself
3	Write Terraform for the S3 resume bucket — versioning enabled, server-side encryption (SSE-S3), block public access	S3 security config is bread and butter for cloud

#	Task	Why You're Doing This
		engineers
4	Write the IAM role + policy for the resume-upload Lambda — scoped to <code>s3:PutObject</code> on the specific bucket only	Least-privilege IAM is THE skill. Write every policy by hand
5	Write the Lambda function handler in Python — accept a base64-encoded PDF via API Gateway, upload to S3 with a unique key (e.g., <code>resumes/{user_id}/{timestamp}.pdf</code>)	This is core boto3 — you need to know <code>s3.put_object()</code> cold
6	Write the API Gateway REST API in Terraform — <code>POST /resume</code> endpoint wired to the Lambda	API Gateway config in Terraform is painful but essential to learn
7	Test the full flow manually — <code>curl</code> a base64 PDF to your API endpoint, verify it lands in S3	Manual testing builds debugging instincts
8	Set up GitHub Actions CI/CD — <code>terraform fmt</code> , <code>terraform validate</code> , <code>terraform plan</code> on PR, <code>terraform apply</code> on merge to <code>main</code>	You already have experience here from aws-cloud-labs, but do it fresh

CLAUDE CODE Does

#	Task	Why Delegating This
1	Generate the <code>package.json</code> , ESLint config, Prettier config for the React frontend	Boilerplate config isn't teaching you anything
2	Scaffold the basic React app structure (folders, routing setup, placeholder pages)	You'll customize it later, structure is just scaffolding
3	Build the resume upload UI component — drag-and-drop zone, file validation (PDF only, <5MB), upload progress bar	Frontend polish isn't your learning target
4	Write unit tests for the Lambda handler (mocking S3 with <code>moto</code>)	Once you've written the handler, test generation is tedious

✅ Phase 1 Deliverable

A working pipeline: user uploads a PDF → hits API Gateway → Lambda stores it in S3. Frontend has a basic upload page.

Phase 2: Resume Parsing & User Profile

Goal: Parse the uploaded resume to extract skills, experience, and preferences. Store a user profile in DynamoDB.

YOU Do (Manually)

#	Task	Why You're Doing This
1	Design the DynamoDB <code>UserProfiles</code> table schema — decide on PK (<code>user_id</code>), attributes (skills, experience_years, target_roles, location_pref, salary_range)	Table design is a core DynamoDB skill — think about access patterns FIRST
2	Write the Terraform for the DynamoDB table — including provisioned capacity or on-demand decision	You need to understand capacity modes and cost implications
3	Write the resume-parsing Lambda in Python — use <code>pdfplumber</code> or <code>PyPDF2</code> to extract text, then parse out skills, job titles, years of experience	This is real Python problem-solving — string parsing, regex, logic
4	Write the boto3 code to store the parsed profile in DynamoDB (<code>put_item</code>)	You need to know DynamoDB item operations inside out
5	Write the IAM policy for this Lambda — <code>s3:GetObject</code> on the resume bucket + <code>dynamodb:PutItem</code> on the profiles table	Again, hand-craft every policy
6	Wire up S3 event notification → Lambda trigger (when a new resume lands in S3, auto-trigger parsing)	Event-driven architecture is a key serverless pattern
7	Add the API Gateway endpoint <code>GET /profile/{user_id}</code> to retrieve the parsed profile	Practice the read path too

CLAUDE CODE Does

#	Task	Why Delegating This
1	Build the "preference questionnaire" UI — a clean multi-step form (target role, location, remote/hybrid, salary range)	Form UI is frontend work
2	Generate a skills taxonomy/keyword list (cloud engineering terms, AWS services, tools, certs) for the parser to match against	Data compilation is tedious

#	Task	Why Delegating This
3	Write the frontend profile display component — show parsed skills as tags, experience summary, editable preferences	UI rendering

✅ Phase 2 Deliverable

Upload a resume → auto-parsed into a structured profile → stored in DynamoDB → visible on the dashboard with editable preferences.

Phase 3: Job Aggregation Engine

Goal: Build scheduled scrapers/API integrations that pull cloud engineering jobs from public sources into DynamoDB.

🔧 YOU Do (Manually)

#	Task	Why You're Doing This
1	Design the DynamoDB <code>Jobs</code> table — PK (<code>job_id</code>), attributes (title, company, location, remote, salary, skills_required, source, url, posted_date, scraped_at). Think about GSIs for querying by location, date, etc.	This table is more complex — GSI design matters
2	Write the Terraform for the Jobs table + GSIs	Terraform GSI syntax is tricky, learn it
3	Write a Lambda function that calls the Adzuna API (free tier, good for job data) — fetch cloud/AWS jobs, transform the response, store in DynamoDB	API integration + data transformation in Python
4	Write a second Lambda for Remotive API (remote jobs, free, no auth)	Practice the pattern again with a different API shape
5	Write the EventBridge rule in Terraform — trigger these Lambdas on a cron schedule (e.g., daily at 6 AM UTC)	Scheduled events are a real-world serverless pattern
6	Write the IAM policies for both scraper Lambdas	You know the drill
7	Write a <code>GET /jobs</code> API endpoint with query params (location, remote, date range) — read from DynamoDB with proper <code>query</code> or <code>scan</code>	DynamoDB query vs scan is an interview favorite

#	Task	Why You're Doing This
	operations	
8	Handle deduplication — same job from multiple sources shouldn't create duplicate entries	Real data engineering problem

CLAUDE CODE Does

#	Task	Why Delegating This
1	Build the job listings UI — card layout with company, title, location, salary, tags, "Apply" button	Frontend display
2	Build filter/sort UI — dropdowns for location, remote, date posted, salary range	Interactive UI components
3	Generate the API response transformation functions (normalize different API response schemas into your DynamoDB schema)	Data mapping is repetitive once you understand the pattern
4	Write integration tests for the scraper Lambdas	Test boilerplate

Phase 3 Deliverable

Jobs auto-scraped daily from multiple sources → stored in DynamoDB → browsable on the dashboard with filters.

Phase 4: AI-Powered Job Matching

Goal: Score and rank jobs against the user's profile using intelligent matching.

YOU Do (Manually)

#	Task	Why You're Doing This
1	Design the matching algorithm — start simple: keyword overlap between user skills and job requirements, weighted by importance	Algorithm design is your brain work
2	Write the matching Lambda in Python — take a user profile + batch of jobs → return scored/ranked results	Core logic, must be yours
3	Store match scores in DynamoDB (or as a GSI sort key) so you can query "top matches for user X" efficiently	DynamoDB modeling for sorted access patterns

#	Task	Why You're Doing This
4	Add the <code>GET /matches/{user_id}</code> API endpoint	Another API Gateway route
5	(Stretch) Integrate Claude API for smarter matching — send the resume text + job description, ask Claude to score fit on a 1-100 scale with reasoning	Using an LLM API in a Lambda is a great talking point
6	Write IAM policy for the matching Lambda (DynamoDB read on both tables, optionally external API call to Anthropic)	Policy writing

CLAUDE CODE Does

#	Task	Why Delegating This
1	Build the "Top Matches" dashboard view — ranked cards with match score, match reasons, skill overlap visualization	Frontend visualization
2	Build a "Why this match?" expandable section on each card showing which skills aligned	UI component
3	Generate the prompt template for Claude API matching (if you go the LLM route)	Prompt engineering is fast with Claude Code

Phase 4 Deliverable

Personalized job rankings based on resume match. Dashboard shows top matches with scores and reasoning.

Phase 5: Application Tracking & Polish

Goal: Track which jobs you've applied to, saved, or dismissed. Polish the full experience.

YOU Do (Manually)

#	Task	Why You're Doing This
1	Design the <code>Applications</code> DynamoDB table — PK (<code>user_id</code>), SK (<code>job_id</code>), attributes (status: saved/applied/rejected/interview, applied_date, notes)	Another table design exercise
2	Write the Terraform for the table	Consistency

#	Task	Why You're Doing This
3	Write <code>POST /applications</code> and <code>PATCH /applications/{job_id}</code> Lambda handlers	CRUD operations in Lambda
4	Add CloudWatch alarms — Lambda errors, DynamoDB throttling, API Gateway 5xx rates	Monitoring is a real-world must-have
5	Set up CloudFront distribution for the React frontend (S3 origin, custom domain if you want)	You did this with Cricket Zone — reinforce the pattern
6	Write a comprehensive README for the repo — architecture diagram description, setup instructions, design decisions	You write this yourself so you can articulate it in interviews

 CLAUDE CODE Does

#	Task	Why Delegating This
1	Build the application tracker UI — Kanban board (Saved → Applied → Interview → Offer/Rejected)	Complex UI component
2	Build the full dashboard layout — sidebar nav, responsive design, dark mode toggle	Frontend fit and finish
3	Generate an architecture diagram (Mermaid or SVG) for the README	Visual asset
4	Write unit + integration tests for all remaining Lambda handlers	Test coverage
5	Add loading states, error handling, empty states to all UI components	UX polish

 Phase 5 Deliverable

Full working application: upload resume → see matched jobs → track applications → polished dashboard.

Architecture Overview





Job Data Sources (Free/Public APIs)

Source	Type	Auth	Notes
Adzuna API	REST API	Free API key	Great coverage, supports location + keyword filters
Remotive	REST API	None	Remote-only jobs, good for cloud roles
GitHub Jobs	RSS/API	None	Tech-focused listings

Source	Type	Auth	Notes
JSearch (RapidAPI)	REST API	Free tier	Aggregates LinkedIn, Indeed, Glassdoor (limited calls)
The Muse	REST API	Free	Company profiles + jobs
USAJobs	REST API	Free API key	Government cloud roles (good for UCOP context)

Estimated Timeline

Phase	Duration	Focus
Phase 1	1-2 weeks	Foundation, S3, upload flow
Phase 2	1-2 weeks	Resume parsing, DynamoDB profiles
Phase 3	2-3 weeks	Job scrapers, EventBridge, listings UI
Phase 4	1-2 weeks	Matching algorithm, scoring
Phase 5	1-2 weeks	Tracking, monitoring, polish
Total	6-10 weeks	

Key Interview Talking Points This Project Gives You

1. "I designed DynamoDB table schemas based on access patterns" — GSIs, PK/SK design, query vs scan trade-offs
2. "I wrote least-privilege IAM policies for each Lambda" — real security thinking, not (*) wildcards
3. "I built event-driven architecture with S3 triggers and EventBridge schedules" — serverless design patterns
4. "I managed all infrastructure with Terraform including remote state" — IaC maturity
5. "I integrated multiple external APIs and normalized the data" — real-world data engineering

6. "I implemented CI/CD with GitHub Actions for Terraform" — DevOps practices
 7. "I set up CloudWatch alarms for operational monitoring" — production readiness
 8. "I used Claude's API for AI-powered job matching" — modern AI integration
-

Quick-Start Checklist (Do This First)

- ☐ Create `cloudhunt` repo on GitHub
- ☐ Set up Terraform remote state (S3 bucket + DynamoDB lock table)
- ☐ Write initial `main.tf` with AWS provider
- ☐ Create S3 resume bucket with encryption
- ☐ Write and deploy first Lambda (resume upload)
- ☐ Test with `curl`
- ☐ Commit and push

Let's build. 🚀